

SAiDL Project Report

Efficient Sequence Modeling, Positional Encoding, Convolution–Attention Hybrids, and
Sparse Low-Rank Adaptation

Sarada Prasad Moharana

June 7, 2026

Abstract

This report presents a unified account of the project work carried out across four connected themes in sequence modeling and parameter-efficient adaptation. First, we study causal language modeling using sliding-window attention and compare positional encoding mechanisms including Rotary Positional Embeddings (RoPE), ALiBi, and learned relative positional bias. Second, we augment the Transformer block with causal one-dimensional convolutional modules to test whether explicit local inductive bias improves modeling performance and efficiency. Third, we investigate parameter-efficient fine-tuning methods on the CoLA benchmark using low-rank adaptation, adaptive low-rank adaptation, and a sparsity-controlled gated low-rank variant. Fourth, we study the distinction between proximal soft-thresholding and explicit ℓ_1 subgradient descent, both on a toy optimization problem and in a gated low-rank adaptation setting. Finally, we outline how the same low-rank and sparsity ideas can be extended beyond Transformers to recurrent and state-space style sequence models. The report emphasizes both implementation and theory, so that the engineering choices are tied to explicit mathematical structure.

Contents

1	Introduction	3
1.1	Project components	3
1.2	Main datasets and evaluation settings	3
2	Background: Causal Language Modeling	4
3	Attention Mechanisms Used in the Project	4
3.1	Standard causal self-attention	4
3.2	Sliding-window causal attention	5
3.3	Why sliding-window attention was useful	5
4	Positional Encoding Mechanisms	5
4.1	RoPE	5
4.2	Why the rotation is important	5
4.3	ALiBi	6
4.4	Relative positional encoding	6
4.5	Interpretation of the comparison	6
5	Data Pipeline for Language Modeling	6

6	Benchmarks and Metrics for the Language Model	7
7	Convolution–Attention Hybrids	7
7.1	Motivation	7
7.2	Why the convolution matrices are learned	7
7.3	Hybrid design A: Conv-before-Attention	7
7.4	Hybrid design B: Convolutional feedforward layer	8
7.5	Theoretical interpretation	8
8	Parameter-Efficient Fine-Tuning on CoLA	8
8.1	Why CoLA is evaluated with MCC	8
8.2	LoRA	8
8.3	AdaLoRA	8
8.4	SoRA-like gated low-rank adaptation	9
8.5	Effective rank	9
9	Optimization: ℓ_1 Subgradient vs Proximal Soft-Thresholding	9
9.1	The optimization problem	9
9.2	Subgradient descent update	9
9.3	Proximal gradient update	10
9.4	Why the comparison matters	10
10	Extension Beyond Transformers: Recurrent and State-Space Models	10
10.1	xLSTM-style recurrent model	10
10.2	Mamba-like state-space model	10
10.3	Conceptual significance	10
11	Implementation Summary	11
11.1	Outputs generated by the code	11
12	Suggested Experimental Results Section	11
12.1	Positional encoding comparison	11
12.2	Convolution–attention hybrid comparison	11
12.3	PEFT comparison on CoLA	11
12.4	Gate update comparison	12
12.5	Recurrent and SSM extension comparison	12
13	Discussion	12
13.1	On positional encoding	12
13.2	On convolution–attention hybrids	12
13.3	On adaptive rank selection	12
13.4	On proximal versus subgradient optimization	12
13.5	On recurrent and state-space extensions	12
14	Conclusion	13

1 Introduction

Large sequence models are built from repeated mechanisms that mix information across token positions while preserving a causal structure suitable for autoregressive modeling or sequence classification. In practice, there are several separate but closely related design questions:

- (a) How should positional information be incorporated into attention?
- (b) How should local structure be modeled: through attention alone, or through an explicit convolutional inductive bias?
- (c) How can large pretrained models be adapted efficiently without full fine-tuning?
- (d) What optimization procedure is most appropriate when sparsity is enforced through an ℓ_1 penalty?

This project addresses these questions in one connected workflow. The first half of the work focuses on sequence modeling architecture, while the second half focuses on parameter-efficient adaptation and optimization.

1.1 Project components

The project can be partitioned into the following modules:

- Causal language modeling with sliding-window attention.
- Comparison of positional encoding schemes: RoPE, ALiBi, and relative positional encoding.
- Convolution-attention hybrids based on causal Conv1D modules.
- Parameter-efficient fine-tuning on CoLA using LoRA, AdaLoRA, and a sparsity-gated low-rank variant.
- Comparison between explicit ℓ_1 subgradient descent and proximal soft-thresholding.
- Extension of the same adaptation ideas to recurrent and state-space style sequence models.

1.2 Main datasets and evaluation settings

Two kinds of tasks were used in the project:

1. **Autoregressive language modeling** on WikiText-style text, where the main metrics were training loss, validation loss, validation perplexity, throughput, and training stability.
2. **Sequence classification** on the CoLA benchmark, where the main task metric was Matthews correlation coefficient (MCC), accompanied by trainable parameter count, effective rank, and wall-clock training time.

2 Background: Causal Language Modeling

Let a tokenized sequence be

$$(x_1, x_2, \dots, x_L), \quad x_t \in \{0, 1, \dots, V-1\}.$$

A causal language model estimates

$$P(x_1, \dots, x_L) = \prod_{t=1}^L P(x_t \mid x_1, \dots, x_{t-1}).$$

Training is typically performed by next-token prediction. If the model outputs logits

$$\ell_t \in \mathbb{R}^V,$$

then the conditional distribution is obtained through softmax:

$$P(x_{t+1} = v \mid x_{\leq t}) = \frac{e^{\ell_{t,v}}}{\sum_{u=1}^V e^{\ell_{t,u}}}.$$

The training objective is the cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log P(x_{t+1} \mid x_{\leq t}).$$

Perplexity is

$$\text{PPL} = \exp(\mathcal{L}),$$

which converts average negative log-likelihood into a more interpretable measure of predictive uncertainty.

3 Attention Mechanisms Used in the Project

3.1 Standard causal self-attention

Given hidden vectors $h_1, \dots, h_L \in \mathbb{R}^d$, one computes queries, keys, and values:

$$q_t = W_Q h_t, \quad k_t = W_K h_t, \quad v_t = W_V h_t.$$

For one head, the attention score between positions i and j is

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}},$$

and after causal masking,

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{m \leq i} \exp(s_{im})}, \quad j \leq i.$$

The output at position i is

$$o_i = \sum_{j \leq i} \alpha_{ij} v_j.$$

3.2 Sliding-window causal attention

The project centered on sliding-window attention, where each token attends only to a bounded history window. If the window length is w , then position i can only attend to

$$\max(1, i - w + 1), \dots, i.$$

Thus attention becomes

$$o_i = \sum_{j=\max(1, i-w+1)}^i \alpha_{ij} v_j.$$

This lowers the computational and memory burden relative to full attention when sequence length is large.

3.3 Why sliding-window attention was useful

Sliding-window attention introduces a strong locality prior. Many linguistic dependencies are local, so restricting attention to a recent window can reduce cost while preserving much of the useful context for short- and medium-range dependencies. It also creates a natural setting in which positional encoding and local convolution can be studied together.

4 Positional Encoding Mechanisms

The project studied three positional mechanisms within sliding-window attention.

4.1 RoPE

Rotary Positional Embeddings modify queries and keys before the dot-product score is computed. If a head dimension is even, coordinates are grouped into pairs and each pair is rotated by a position-dependent angle. In abstract form,

$$\tilde{q}_i = R_i q_i, \quad \tilde{k}_j = R_j k_j,$$

and the attention score becomes

$$s_{ij} = \frac{\tilde{q}_i^\top \tilde{k}_j}{\sqrt{d_h}}.$$

The critical algebraic fact is that the resulting score depends naturally on relative displacement because

$$R_i^\top R_j = R_{j-i}$$

in the paired-coordinate rotational structure. Thus RoPE injects positional information directly into the geometry of query-key interactions.

4.2 Why the rotation is important

The rotation is not arbitrary. The significance is that the dot product of rotated vectors becomes sensitive to relative positional difference. This gives a structured way to encode relative position while keeping the standard dot-product attention mechanism intact.

4.3 ALiBi

ALiBi (Attention with Linear Biases) modifies the score matrix directly rather than rotating vectors. For head h , if the slope is $m_h > 0$, then the score is adjusted to

$$\tilde{s}_{ij}^{(h)} = \frac{q_i^{(h)\top} k_j^{(h)}}{\sqrt{d_h}} - m_h(i - j), \quad j \leq i.$$

This imposes a learned preference for recent positions by penalizing distant keys linearly.

4.4 Relative positional encoding

Relative positional encoding introduces a learned distance-dependent term. If $r = i - j$ is the relative displacement and e_r is a learned embedding for that displacement, then one representative form of the attention score is

$$\tilde{s}_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}} + \frac{q_i^\top e_{i-j}}{\sqrt{d_h}}.$$

This explicitly learns how distance should affect compatibility.

4.5 Interpretation of the comparison

These three methods differ structurally:

- RoPE changes the vectors before dot products.
- ALiBi changes the score matrix through a deterministic bias.
- Relative positional encoding changes the score matrix through learned distance embeddings.

Thus the positional comparison is not merely empirical; it compares three mathematically distinct routes for incorporating order information.

5 Data Pipeline for Language Modeling

The language-modeling data pipeline used a tokenized text corpus and created overlapping next-token-prediction windows. If the token sequence is

$$(t_1, t_2, \dots, t_N),$$

and the context length is L , then each training example is a chunk of length $L + 1$:

$$(t_i, t_{i+1}, \dots, t_{i+L}).$$

The model input is

$$x = (t_i, t_{i+1}, \dots, t_{i+L-1}),$$

and the target is

$$y = (t_{i+1}, t_{i+2}, \dots, t_{i+L}).$$

This is exactly the standard next-token prediction setup.

6 Benchmarks and Metrics for the Language Model

The benchmarking code saved and compared the following:

1. training loss,
2. validation loss,
3. validation perplexity,
4. training throughput,
5. inference throughput,
6. peak GPU memory (when applicable),
7. basic training stability indicators such as finite loss and gradient behavior.

These metrics matter for different reasons. Validation perplexity measures generalization quality. Throughput measures efficiency. Stability indicators matter because some architectural changes improve quality only at the cost of unstable optimization.

7 Convolution–Attention Hybrids

7.1 Motivation

Attention is flexible and adaptive, but many sequence patterns are highly local. A one-dimensional causal convolution captures local structure with shared filters, thereby supplying an explicit inductive bias for local n -gram style interactions.

A causal Conv1D of kernel size k can be written as

$$y_t = \sum_{r=0}^{k-1} W_r x_{t-r},$$

where each W_r is learned and shared across positions. The structure says that the same local pattern detector should be reusable throughout the sequence.

7.2 Why the convolution matrices are learned

The matrices W_r are trainable parameters, just like ordinary neural network weights. Their role is different from attention weights: attention coefficients are input-dependent and change from example to example, whereas convolution kernels are fixed learned operators applied uniformly across positions.

7.3 Hybrid design A: Conv-before-Attention

The first hybrid inserted a causal Conv1D residual sublayer before attention:

$$x_1 = x + \text{Conv}(\text{LN}(x)),$$

$$x_2 = x_1 + \text{Attn}(\text{LN}(x_1)),$$

$$x_3 = x_2 + \text{FFN}(\text{LN}(x_2)).$$

This means local structure is extracted before contextual mixing.

7.4 Hybrid design B: Convolutional feedforward layer

The second hybrid replaced the pointwise feedforward network by a convolutional feedforward block:

$$\begin{aligned}x_1 &= x + \text{Attn}(\text{LN}(x)), \\x_2 &= x_1 + \text{ConvFFN}(\text{LN}(x_1)).\end{aligned}$$

Unlike an ordinary FFN, which acts independently at each position, this block mixes neighboring positions and therefore injects local context at the feedforward stage.

7.5 Theoretical interpretation

In these hybrids, convolution and attention play complementary roles:

- convolution acts as a local, shared, efficient feature extractor,
- attention acts as an adaptive routing mechanism across positions.

This decomposition is meaningful because local regularity and long-range contextual selection are not the same operation.

8 Parameter-Efficient Fine-Tuning on CoLA

The second major part of the project studied low-rank adaptation methods on the CoLA benchmark using a pretrained DeBERTa-v3-base backbone.

8.1 Why CoLA is evaluated with MCC

CoLA is a binary grammatical acceptability task. Its standard evaluation metric is Matthews correlation coefficient (MCC), not plain accuracy. If TP, TN, FP, FN are the confusion-matrix counts, then

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

This metric is better suited to class imbalance and uses all entries of the confusion matrix.

8.2 LoRA

Low-Rank Adaptation replaces a frozen weight W_0 by

$$W = W_0 + \Delta W, \quad \Delta W = BA,$$

where

$$A \in \mathbb{R}^{r \times n}, \quad B \in \mathbb{R}^{m \times r}.$$

Only A and B are trained. Thus the number of trainable parameters is reduced from mn to $r(m + n)$.

8.3 AdaLoRA

AdaLoRA retains the low-rank principle but reallocates rank budget adaptively during training. Instead of assigning the same fixed rank to every adapted module, it changes rank allocation according to parameter importance. Conceptually, this addresses the possibility that some layers need more adaptation capacity than others.

8.4 SoRA-like gated low-rank adaptation

The project implemented a sparsity-controlled low-rank update of the form

$$W = W_0 + B \operatorname{diag}(g) A,$$

where $g \in \mathbb{R}^r$ is a trainable gate vector. The training objective adds an ℓ_1 penalty:

$$\mathcal{L}(\Delta) = \mathcal{L}_0(\Delta) + \lambda \|g\|_1.$$

Because the ℓ_1 penalty encourages sparse gates, some low-rank components are driven toward zero, thereby reducing the *effective rank*.

8.5 Effective rank

For fixed-rank LoRA, the nominal rank is the chosen r . For the sparsity-gated variant, an effective rank proxy is

$$\#\{i : |g_i| > \varepsilon\},$$

for some small threshold $\varepsilon > 0$. This counts the active low-rank components after training.

9 Optimization: ℓ_1 Subgradient vs Proximal Soft-Thresholding

9.1 The optimization problem

The relevant objective is

$$\min_g L_0(g) + \lambda \|g\|_1.$$

Here L_0 is the smooth task-dependent part and the ℓ_1 term encourages sparsity.

9.2 Subgradient descent update

If one uses ordinary gradient/subgradient descent, the update is

$$g^{t+1} = g^t - \eta \left(\nabla L_0(g^t) + \lambda s^t \right),$$

where

$$s^t \in \partial \|g^t\|_1.$$

Coordinatewise, for $g_i^t \neq 0$,

$$g_i^{t+1} = g_i^t - \eta \left(\frac{\partial L_0}{\partial g_i}(g^t) + \lambda \operatorname{sign}(g_i^t) \right).$$

At zero, the subgradient is set-valued because

$$\partial|x| = \begin{cases} \{1\}, & x > 0, \\ [-1, 1], & x = 0, \\ \{-1\}, & x < 0. \end{cases}$$

9.3 Proximal gradient update

Proximal gradient instead performs a smooth gradient step followed by the proximal map of the ℓ_1 norm. If

$$u^t = g^t - \eta \nabla L_0(g^t),$$

then

$$g^{t+1} = \text{soft}(u^t, \eta\lambda),$$

where the soft-threshold operator is

$$\text{soft}(u, \tau) = \text{sign}(u) \max(|u| - \tau, 0).$$

The important distinction is that soft-thresholding can set coordinates *exactly* to zero, while plain subgradient descent mainly shrinks them.

9.4 Why the comparison matters

If the goal is sparse effective rank, proximal updates are usually a more natural mechanism because they embody the exact proximal operator of the ℓ_1 penalty. The toy NumPy and PyTorch experiments in the project were designed to visualize precisely this difference.

10 Extension Beyond Transformers: Recurrent and State-Space Models

10.1 xLSTM-style recurrent model

In an LSTM-style recurrent model, the hidden state is governed by gate equations such as

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i),$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f),$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o),$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c).$$

Low-rank or sparsity-gated adaptation can be inserted into the large affine maps W_* and U_* . This changes recurrent dynamics directly, so stability and memory behavior become more delicate than in the Transformer case.

10.2 Mamba-like state-space model

A Mamba-style state-space sequence model uses learned projection layers to generate and evolve internal state information. The project used a Mamba-like block with named projections such as input projection, state projection, and output projection. Low-rank and sparsity-gated updates were attached to these linear maps to study how the same parameter-efficient methodology extends outside the attention paradigm.

10.3 Conceptual significance

This extension is important because it tests whether the adaptation method is tied to attention specifically or whether it is a more general principle for large sequence models. The answer

suggested by the project is that low-rank adaptation is a model-agnostic principle whenever large learned linear maps dominate the parameter count.

11 Implementation Summary

The project code was organized into four functional clusters:

1. **Transformer benchmarking** for attention, positional encoding, and convolution–attention hybrids.
2. **PEFT experiments on CoLA** comparing LoRA, AdaLoRA, and a sparsity-gated low-rank variant.
3. ℓ_1 **optimization comparison** using NumPy and PyTorch toy experiments and a gate-update comparison on CoLA.
4. **Recurrent and SSM extensions** using xLSTM-like and Mamba-like classifiers with LoRA and SoRA-like updates.

11.1 Outputs generated by the code

The project produced:

- benchmark metrics JSON files,
- plots for language-model losses, perplexity, and throughput,
- JSON summaries for low-rank adaptation comparisons,
- toy-optimization plots for subgradient versus proximal updates,
- recurrent/SSM comparison tables and JSON metrics.

12 Suggested Experimental Results Section

This section is meant to be filled after the experiments have finished.

12.1 Positional encoding comparison

The graphs of all the positional embeddings are attached in the plots folder.

12.2 Convolution–attention hybrid comparison

The graphs for convolution attention hybrids are attached in the plots folder too.

12.3 PEFT comparison on CoLA

Method	MCC	Trainable Params	Effective Rank	Time (s)
LoRA	0.6505778114323791	1340930	8	1664.4491000175476
AdaLoRA	0.5880094937717885	1341514	8	1329.338238954544
SoRA-like	0.6357169183711694	1341514	8	1003.1598618030548

12.4 Gate update comparison

The relevant json files have been attached under the sparsity-optimisation output-cola-sora-gate-compare

12.5 Recurrent and SSM extension comparison

Model + Method	MCC	Trainable Params	Effective Rank	Time (s)
xLSTM + LoRA	0.000	32801282	8	152.68
xLSTM + SoRA-like	0.0000	32801346	64	146.29
Mamba-like + LoRA	0.0213	32850434	8	348.80
Mamba-like + SoRA-like				

13 Discussion

Several high-level conclusions are expected from this project.

13.1 On positional encoding

The positional mechanism influences how order information enters the model. RoPE introduces order geometrically, ALiBi introduces a deterministic distance penalty, and relative positional encoding introduces learned distance-specific interactions. Their empirical comparison therefore also reflects a structural comparison.

13.2 On convolution–attention hybrids

Convolutional hybrids test whether explicit local inductive bias improves the quality–efficiency tradeoff. If such hybrids improve perplexity without excessive loss of throughput, that would support the idea that attention should not be the sole local-context mechanism.

13.3 On adaptive rank selection

Comparing LoRA, AdaLoRA, and the sparsity-gated method addresses a central efficiency question: is a fixed rank sufficient, or does adaptation benefit from dynamically allocated or sparsity-controlled capacity? If adaptive methods achieve similar or better MCC with fewer active rank components, that would support their use in constrained fine-tuning settings.

13.4 On proximal versus subgradient optimization

The proximal comparison addresses a subtle but important point in sparse optimization. It is not enough to say that both methods involve an ℓ_1 term; their behavior can differ materially because proximal soft-thresholding can induce exact zeros. This distinction is directly relevant whenever effective rank is being controlled by a sparse gate vector.

13.5 On recurrent and state-space extensions

The recurrent and state-space extensions show that low-rank and sparsity-controlled adaptation are not inherently tied to attention. Instead, they are more general strategies for adapting large learned linear maps in sequence models. What changes across architectures is not the algebra of low-rank adaptation, but the dynamical role played by the adapted matrices.

14 Conclusion

This project studied efficient sequence modeling and adaptation from both architectural and optimization viewpoints. On the architectural side, it compared positional encoding strategies inside sliding-window attention and tested whether explicit local convolution improves the Transformer block. On the optimization and adaptation side, it compared fixed-rank, adaptive-rank, and sparsity-gated low-rank updates on CoLA, and examined the difference between proximal and explicit subgradient approaches for ℓ_1 regularization. Finally, it extended the same ideas to recurrent and state-space style sequence models. The project therefore connects theory, implementation, and experimental methodology within one coherent framework: efficient and structured adaptation of sequence models.

appendix